AI-Powered Skill Matching Recruitment System

SOA Project Review-2

Date& Time
Thursday, 24th September 2026

Under the Guidance of
Dr.Neelima Gogineni
Associate Professor,
Department of Computer Science and Engineering,
KLH CSE Bowrampet Campus

Team Members
K. Nikhil sai - 2420030635
Santosh - 2420030198
Pranav Itharaju- 2420030538

Outline of Project Work

- Problem Statement & Objectives
- Project Design
- System Architecture
- Challenges
- Technology Stack
- Detailed Technical Design
- Implementation
- Technical Soundness
- Current Status and Future Work

Problem Statement & Objectives

Problem Statement

- • Manual and time-consuming resume screening
  • Difficulty in matching candidate skills with job requirements
  • Exact keyword matching missing related skills and terminology
  • Unstructured information across resumes and job descriptions
  • Difficulty identifying missing or required skills
  • Inconsistent candidate comparison and ranking
  • Limited explanation of candidate–job compatibility
  • Difficulty handling large numbers of candidate profiles

Proposed Solution

AI-Powered Skill Matching Recruitment System provides an intelligent recruitment platform using **NLP, semantic skill matching**

Objectives

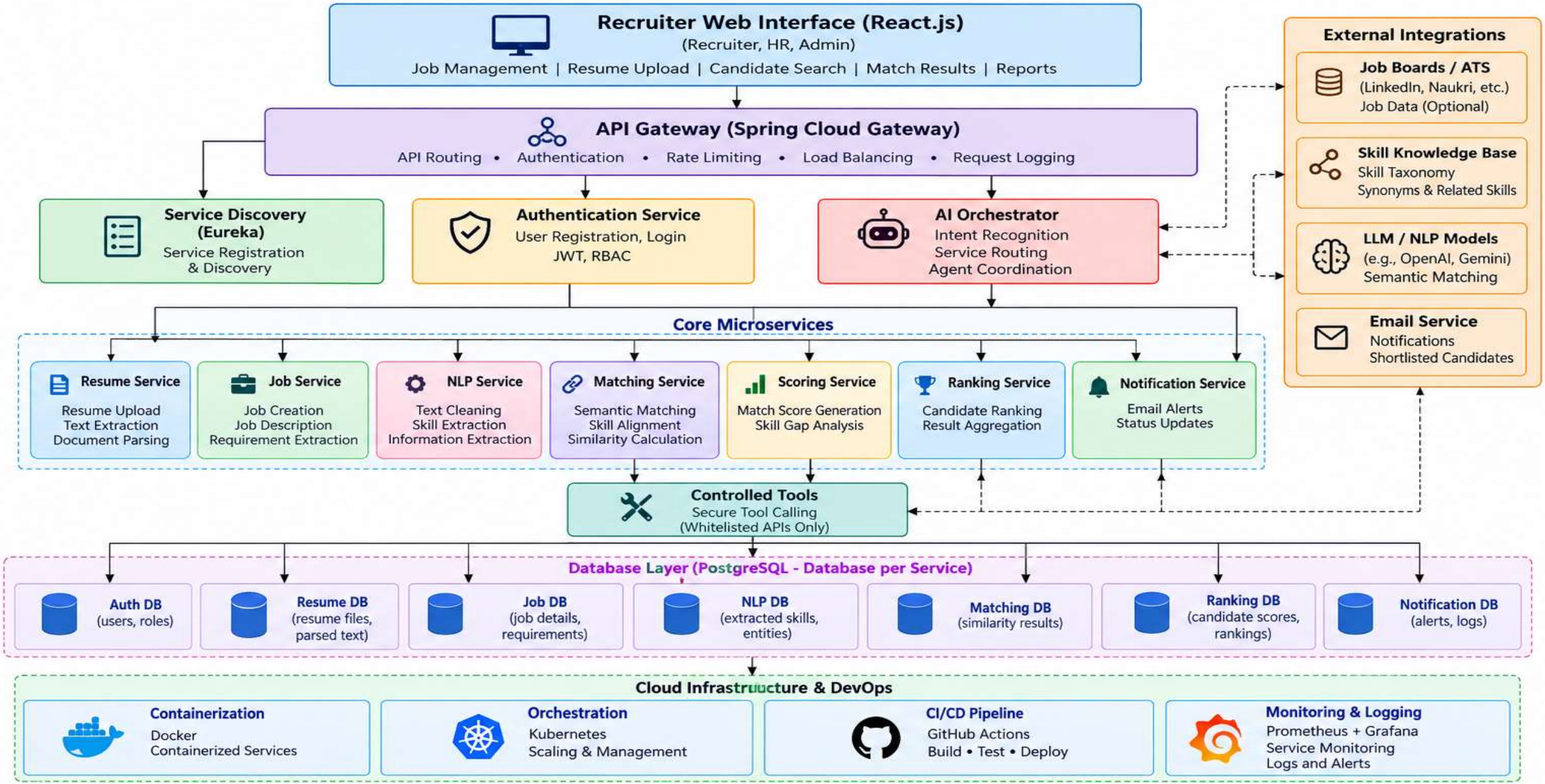
- Develop an intelligent **NLP-based recruitment matching system**.
  • Extract **skills, experience, education, projects and certifications** from resumes.
  • Extract **required skills and qualifications** from job descriptions.
  • Implement **semantic matching** to identify related skills and terminology.
  • Calculate an interpretable **candidate–job match score**.
  • Identify **matched skills and missing skills** for each candidate.

Project Design

- **React.js** — User interface
- **JWT + RBAC** — Authentication and authorization
- **Spring Cloud Gateway** — API routing
- **Eureka** — Service discovery
- **Spring Boot Microservices** — Recruitment business functionality
- **PostgreSQL** — Service-specific databases
- **Resume Processing Service** — Resume upload and text extraction
- **NLP Service** — Skill and requirement extraction
- **Semantic Matching Service** — Candidate–job skill matching
- **Scoring & Ranking Service** — Match score calculation and candidate ranking
- **Docker/Kubernetes** — Deployment and orchestration

System Architecture

DEEMED TO BE UNIVERSITY



Module Design

Module	Function
Authentication Service	User registration, login, JWT and roles
Resume Service	Resume upload, parsing and candidate information
Job Service	Job creation and job requirement management
NLP Service	Text cleaning, skill extraction and information extraction
Matching Service	Semantic skill matching and similarity calculation
Scoring Service	Match score generation and skill-gap analysis
Ranking Service	Candidate ranking and result aggregation
Notification Service	Recruiter alerts and candidate status updates
Database Service	Candidate, job, skill and match-result storage
AI Service	Intelligent recruitment assistance

AI Agents

- Resume Screening Agent
- Job Analysis Agent
- Skill Matching Agent
- Candidate Ranking Agent
- Skill Gap Agent
- Recruiter Assistant Agent
- Notification Agent

Methodology Adopted

System Methodology

USER
(Recruiter)
↓
React.js Interface
↓
Authentication + JWT
↓
API Gateway
↓
Service Discovery
(Eureka)
↓
Required Microservice
(Resume, Job, Matching, etc.)
↓
Business Logic
↓
Service Database
(PostgreSQL)
↓
Response
↓
React.js Interface

AI Methodology

Resume / Job Description
(AI REQUEST)
↓
Text Preprocessing
(Cleaning & Normalization)
↓
Information Extraction
(Skills, Experience, Education, etc.)
↓
Semantic Representation
(Embeddings)
↓
Semantic Skill Matching
(Similarity Calculation)
↓
Match Score Calculation
(Skill Alignment, Experience, etc.)
↓
Candidate Ranking
(Result Aggregation)
↓
Response
(Matched Skills, Missing Skills, Score)

Supporting Points

- User sends resume and job description through React.js.
- JWT validates the authenticated user.
- Gateway routes the request.
- Eureka identifies the required service.
- Microservice executes the business logic.
- Service accesses its own PostgreSQL database.
- Resume and job description are processed using NLP.
- Skills and requirements are extracted from the text.
- Semantic matching is performed to find related skills.
- Match score is calculated based on multiple factors.
- Candidates are ranked and the final response is returned to the user.
- Results are displayed in the React.js interface.

DEEMED TO BE UNIVERSITY

Technology Stack

Category	Technologies / Tools
Frontend	React.js, Vite, HTML, CSS, JavaScript
Backend	Java 21, Spring Boot, Spring Cloud, REST APIs, OpenFeign
Security	Spring Security, JWT, RBAC, BCrypt
Database	PostgreSQL, Database-per-Service
AI / NLP	Python, NLP Libraries, Semantic Matching, Embeddings (e.g., Sentence Transformers)
Microservices	Spring Boot Microservices, API Gateway, Eureka
DevOps / Cloud	Docker, Kubernetes, AWS, GitHub Actions
Monitoring & Security	Spring Boot Actuator, Prometheus, Grafana, SonarQube, OWASP ZAP

Detailed Technical Design

DEEMED TO BE UNIVERSITY

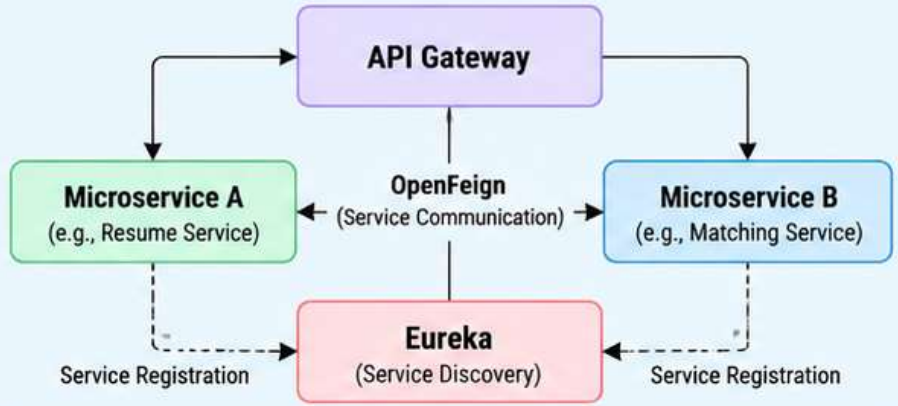
1. Request & Security Flow

- **React Client** (User Interface)
- **API Gateway** (Request Routing)
- **JWT Authentication** (Token Validation)
- **RBAC Authorization** (Role & Access Control)
- **Required Microservice** (Business Logic)
- **PostgreSQL** (Service Database)

2. AI Technical Flow

- **User AI Request** (Resume / Job Description)
- **AI Service** (NLP Processing)
- **AI Orchestrator** (Task Planning & Routing)
- **Intent Recognition** (Identify Task)
- **Agent Selection** (Choose Specialized Agent)
- **Specialized AI Agent** (Resume, Matching, Ranking)
- **Controlled / Whitelisted Tool** (Secure API Calling)
- **Microservice API** (Get / Store Results)
- **AI Response** (Match Results, Explanation)

3. Service Communication



4. Database Design

Authentication Service	Auth DB
Resume Service	Resume DB
Job Service	Job DB
NLP Service	NLP DB
Matching Service	Matching DB
Ranking Service	Ranking DB
Notification Service	Notification DB

Implementation

Backend

Spring Boot Microservices

- Authentication Service
- Resume Service
- Job Description Service
- NLP Processing Service
- Semantic Matching Service
- Scoring & Ranking Service
- REST APIs
- PostgreSQL Integration
- JWT Authentication
- RBAC
- API Gateway
- Eureka Service Discovery
- OpenFeign Communication

Frontend

- Authentication screens
- Registration/Login flow
- Resume Upload Interface
- Job Description Interface
- Candidate Ranking Dashboard

Infrastructure

- Docker configuration
- Kubernetes configuration
- Actuator health monitoring
- GitHub Actions CI/CD

AI

- NLP Service architecture
- Resume text processing
- Skill extraction architecture
- Semantic matching model
- Match scoring architecture
- Candidate ranking model

Technical Soundness

Scalability
- Microservices can be scaled independently.
- Kubernetes can manage multiple service instances.

Security
- JWT provides authenticated API access.
- RBAC restricts functionality according to user roles.
- BCrypt protects stored passwords.
- AI tools are controlled/whitelisted.

Reliability
- Services are independently deployable.
- Service discovery supports distributed communication.
- Health endpoints support service monitoring.

Maintainability
- Each service has a focused responsibility.
- Independent codebases/modules simplify maintenance.
- Database-per-service reduces direct data coupling.

Extensibility
- New microservices can be added independently.
- New AI agents can be introduced without redesigning the entire system.

Performance
- Independent services allow targeted scaling.
- Gateway provides centralized request routing.
- Kubernetes can support resource-based scaling.

Current Status & Future Work

Current Status

Component	Status
Architecture Design	✅ Completed
System Design	✅ Completed
Spring Boot Backend	✅ Implemented
Authentication	✅ Implemented
JWT + RBAC	✅ Integrated
PostgreSQL	✅ Connected
API Gateway	✅ Configured
Core Microservices	✅ Implemented
React.js Frontend	🔄 In Progress
AI Service	🔄 In Progress
AI Agents	🔄 In Progress
Docker	🔄 In Progress
Kubernetes	🔄 In Progress
Complete System Testing	⏳ Pending

Future Work

- Complete remaining healthcare modules.
- Complete AI service and specialized agents.
- Complete frontend dashboards.
- Improve security and error handling.
- Complete integration testing.
- Complete Docker/Kubernetes deployment.
- Improve scalability and performance.
- Enhance monitoring and observability.